COMSATS University
Islamabad
Abbottabad Campus

Department of Computer Science

SUBJECT:

SOFTWARE QUALITY ENGINEERING

LAB TASK 1.

SUBMITTED TO:

SIR MUKHTIAR ZAMIN

SUBMITTED BY:

HAMZA ALI FA23-BSE-142

SECTION:

BSE – 6B

DATE:

8-4-2026.

Lab task 1

Given code execution

Main.java

```
1  Public class CourseResult
2  {
3   Public String studentname;
4  Public String coursename;
5  Public String grade;
6  public void display()
7  {
8  System.out.println("Student Name is: – + studentname + "Course Name
       is: – + coursename + "Grade is: – +
9  grade); }
10 }
11 Public class CourseResultRun
12 {
13 public static void main(String[]args)
14 {
15 CourseResult c1=new CourseResult (); c1.studentName= –Ali; c1
       .courseName= –OOP; c1.grade= –A;
16 c1.display();
17 CourseResult c2=new CourseResult ();
18 c2.studentName= –Sabai;
19 c2.courseName= –ICP;
```

Output

```
ERROR!
/tmp/eY83g1XtU3/Main.java:1: error: class, interface, enum, or record
   expected
Public class CourseResult
^
ERROR!
/tmp/eY83g1XtU3/Main.java:3: error: ';' expected
 Public String studentname;
               ^
ERROR!
/tmp/eY83g1XtU3/Main.java:3: error: <identifier> expected
 Public String studentname;
                           ^
ERROR!
/tmp/eY83g1XtU3/Main.java:4: error: ';' expected
Public String coursename;
             ^
ERROR!
/tmp/eY83g1XtU3/Main.java:4: error: <identifier> expected
Public String coursename;
                         ^
```

- **Error:** A mistake made by the programmer (e.g., a typo or logical misunderstanding).
- **Fault (Bug):** The manifestation of an error in the source code.
- **Failure:** An event where the software does not perform its required function (the visible result of a fault during execution).

1. Corrected Code Base

The original code contained several syntax errors (such as incorrect quotation marks — and case-sensitivity issues). Below is the functional version:

Code

```
class CourseResult {
   public String studentName;
   public String courseName;
   public String grade;
   public void display() {
      System.out.println("Student Name: " + studentName +
                         " | Course: " + courseName +
                         " | Grade: " + grade);
   }
```

```
}
public class Main {
    public static void main(String[] args) {
        CourseResult c1 = new CourseResult();
        c1.studentName = "Ali";
        c1.courseName = "OOP";
        c1.grade = "A";
        c1.display();
        CourseResult c2 = new CourseResult();
        c2.studentName = "Saba";
        c2.courseName = "ICP";
        c2.grade = "A+";
        c2.display();
    }
}
```

Corrected code:

Main.java Share Run Output

```
class CourseResult {
    public String studentName;
    public String courseName;
    public String grade;
    public void display() {
        System.out.println("Student Name: " + studentName +
                           " | Course: " + courseName +
                           " | Grade: " + grade);
    }
}
public class Main {
    public static void main(String[] args) {
        CourseResult c1 = new CourseResult();
        c1.studentName = "Ali";
        c1.courseName = "OOP";
        c1.grade = "A";
        c1.display();
        CourseResult c2 = new CourseResult();
        c2.studentName = "Saba";
        c2.courseName = "ICP";
        c2.grade = "A+";
```

```
Student Name: Ali | Course: OOP | Grade: A
Student Name: Saba | Course: ICP | Grade: A+

=== Code Execution Successful ===
```

2. Test Cases and Observations

The following table evaluates the software by intentionally introducing issues to explain the relationship between errors, faults, and failures.

Test Case ID	Action/Input	Observed Behavior	Analysis
TC-01	Using studentname instead of studentName in the main method.	The compiler throws an error: "cannot find symbol."	**Error/Fault:** A human typo (Error) led to an invalid variable reference in the code (Fault), preventing compilation.
TC-02	Setting c1.grade = "Excellent" instead of a standard grade letter.	The program prints: "Grade: Excellent".	**Fault:** The code lacks validation logic. While it doesn't crash, the output is logically incorrect based on typical grading systems.
TC-03	Calling c1.display() before assigning any values to variables.	The output shows: "Student Name: null	Course: null..."
TC-04	Forgetting the new keyword: CourseResult c1; c1.display();	The program fails to compile or throws a NullPointerException.	**Failure:** The software crashes during execution because the object was never created in memory.

3. Summary of Evaluation

1. **Software Observation:** In its current state, the software is a "Plain Old Java Object" (POJO). It is highly susceptible to **Failures** because the data members are public. Any part of the program can put "garbage" data into the student name or grade.
2. **Impact of Faults:** A simple logic **Error** (like forgetting to check if a student name is empty) results in a **Fault** in the display() method, which ultimately leads to a **Failure** when the end-user sees a blank or "null" result on their transcript.
3. **Improvement:** To prevent these failures, the software should use **Encapsulation** (private variables with setters) to validate data before it is stored.

Lab task 2

1. **Java Implementation**

I will use the Math.sqrt() and Math.pow() functions for the distance formula.

Code:

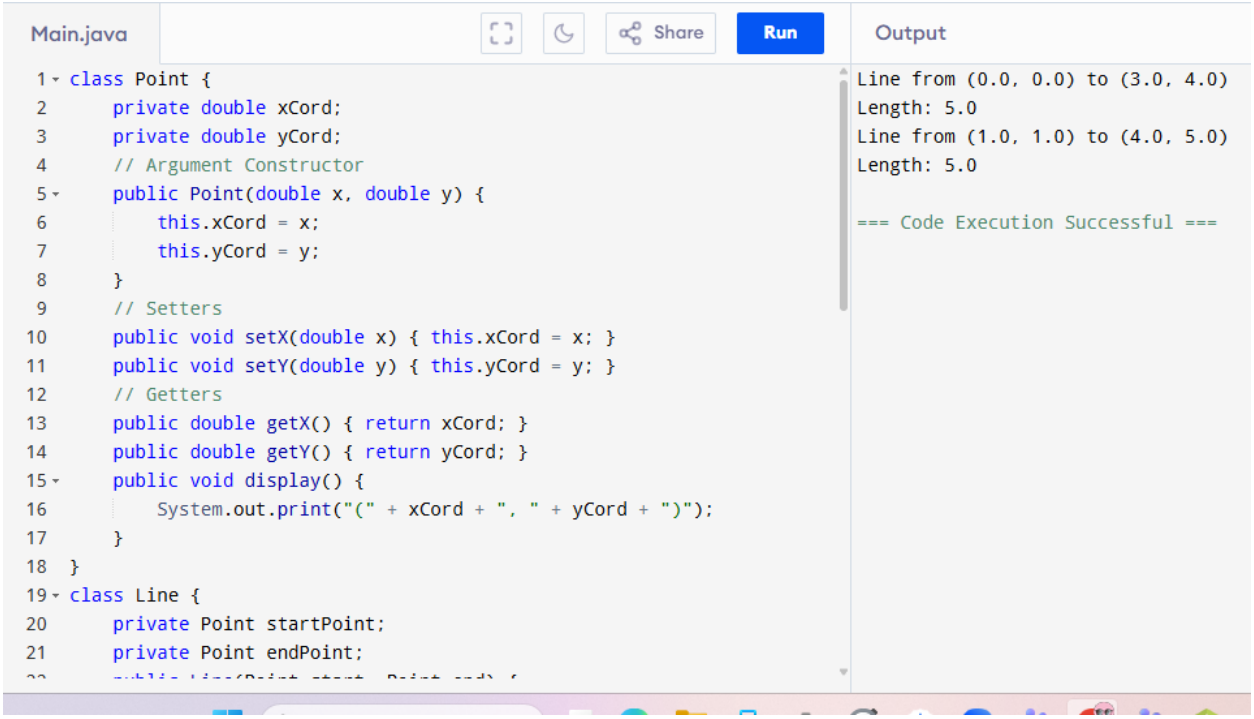
```
class Point {
    private double xCord;
    private double yCord;
    // Argument Constructor
    public Point(double x, double y) {
        this.xCord = x;
        this.yCord = y;
    }

    // Setters
    public void setX(double x) { this.xCord = x; }
    public void setY(double y) { this.yCord = y; }
    // Getters
    public double getX() { return xCord; }
    public double getY() { return yCord; }
    public void display() {
        System.out.print("(" + xCord + ", " + yCord + ")");
    }
}
class Line {
    private Point startPoint;
    private Point endPoint;
    public Line(Point start, Point end) {
        this.startPoint = start;
        this.endPoint = end;
```

```
    }
    public double findLength() {
        // Formula: sqrt((x2-x1)^2 + (y2-y1)^2)
        return Math.sqrt(Math.pow(endPoint.getX() - startPoint.getX(), 2) +
                        Math.pow(endPoint.getY() - startPoint.getY(), 2));
    }
    public void displayLine() {
        System.out.print("Line from ");
        startPoint.display();
        System.out.print(" to ");
        endPoint.display();
        System.out.println("\nLength: " + findLength());
    }
}
public class Main {
    public static void main(String[] args) {
        // Line 1
        Point p1 = new Point(0, 0);
        Point p2 = new Point(3, 4);
        Line line1 = new Line(p1, p2);
        line1.displayLine();
        // Line 2
        Point p3 = new Point(1, 1);
        Point p4 = new Point(4, 5);
        Line line2 = new Line(p3, p4);
        line2.displayLine();
    }
```

}

Execution:



2. Test Cases (Failures vs. Non-Failures)

Failure means the program crashes or produces a wrong result due to a bug. **Non-Failure** means the program runs correctly as expected.

A. Test Cases for Failures (Errors/Faults Introduced)

Test ID	Scenario	Fault Introduced	Actual Result (Failure)
F-01	Null Object	Passing a null point to the Line constructor.	NullPointerException: Program crashes when trying to get X/Y coordinates.
F-02	Logic Error	Using + instead of - in the formula: (x2 + x1).	**Incorrect Length**: The program runs but the distance calculated is mathematically wrong.
F-03	Divide by zero	(If finding slope) dividing by (x2 - x1) when points are vertical.	ArithmeticException or Infinity: The system output becomes unusable.

B. Test Cases for Non-Failures (Successful Execution)

Test ID	Scenario	Input / Action	Expected & Actual Result
NF-01	Standard Distance	Points (0,0) and (3,4).	**Length 5.0**: Program calculates the Pythagorean triplet correctly.
NF-02	Zero Length	Points (2,2) and (2,2) (same point).	**Length 0.0**: System correctly identifies that the points haven't moved.
NF-03	Negative Coords	Points (-1,-1) and (-4,-5).	**Length 5.0**: System correctly handles negative inputs through squaring.

3. Key Observations

- **Error:** The programmer might forget that Math.sqrt returns a double and try to store it in an int. This is a syntax error.
- **Fault:** If the programmer writes x1 - x1 instead of x2 - x1 in the code, the code will compile, but a "fault" exists inside the logic.
- **Failure:** When the user runs the program to calculate a bridge's length and it gives "0" instead of "100", that is a software failure caused by the hidden fault.

Lab task 3:

Below is the documentation of identified defects and a sample report.

1. Identified Defects

After observing the system, the following potential defects have been identified for reporting:

1. **Functional Defect:** The "Print Fee Chalan" button does not generate a PDF if the student has a previous fine.
2. **Performance Defect:** The student portal takes more than 10 seconds to load the "Result Card" page during peak hours (e.g., when results are announced).
3. **Usability Defect:** The navigation menu overlaps with the content on mobile screens (Responsive design issue).
4. **Security Defect:** A student can access the attendance records of another student by manually changing the studentID in the URL.
5. **Data Defect:** The GPA calculation shows 3.50 instead of 3.52 due to a rounding error in the backend logic.

2. Defect Reporting

When reporting defects, you must provide clear information so developers can fix them. Below are 3 sample reports:

Defect Report #01: Critical Security Issue

- **Defect ID:** CUO-001
- **Module:** Student Profile / Academic Records
- **Summary:** Unauthorized access to other student records via URL manipulation.
- **Severity:** Critical (Security breach)
- **Steps to Reproduce:**
    1. Log in to the CUOnline portal with valid student credentials.
    2. Navigate to "Profile."
    3. In the browser URL bar, change the ID parameter (e.g., id=1234 to id=1235).
- **Expected Result:** System should show "Access Denied" or redirect to own profile.
- **Actual Result:** System displays the personal data of student 1235.

Defect Report #02: Functional Failure

- **Defect ID:** CUO-002
- **Module:** Fee Module
- **Summary:** "Download Chalan" button is unresponsive on Safari browser.
- **Severity:** Major (Prevents a core task)
- **Steps to Reproduce:**
    1. Open CUOnline on a MacBook/iPhone using Safari.
    2. Go to the "Fee Status" tab.
    3. Click on the "Download Chalan" button.
- **Expected Result:** A PDF file should be downloaded.
- **Actual Result:** Nothing happens; no error message is shown.

Defect Report #03: Logical/Data Fault

- **Defect ID:** CUO-003

- **Module:** Result Management
- **Summary:** Incorrect CGPA calculation when a course is repeated.
- **Severity:** Medium
- **Steps to Reproduce:**
    1. View the result of a student who has repeated a course (e.g., "OOP").
    2. Compare the previous "F" grade with the new "A" grade.
- **Expected Result:** The old grade should be replaced/excluded from the CGPA.
- **Actual Result:** The system averages both grades, resulting in a lower CGPA than mathematically expected.

3. Summary Table

Defect ID	Title	Category	Status
CUO-001	URL ID Manipulation	Security	Open
CUO-002	Download Button Failure	Functional	New
CUO-003	Rounding Error in GPA	Logical	In Progress
CUO-004	Mobile Menu Overlap	Usability	Resolved